

Data Science – Project Report (Phases 1 and 2)

Information Retrieval Engine

Master 1 – Data Science

Université Paris Cité

Students: AIT MOKHTAR Clara
AYDIN Maria
GOURMELEN Thomas
TAN Vincent

Professors: PALPANAS Themis
CHATZAKIS Manos

Date: 04/04/2026

Contents

1	Introduction	2
1.1	Project Context	2
1.2	Objectives	2
1.3	Scope and Positioning of This Report	2
1.4	Main Contributions	3
2	Dataset Understanding	4
2.1	Data Sources and Schema	4
2.2	Global Dataset Scale	4
2.3	Field Availability and Practical Consequences	4
2.4	Content Construction and Preprocessing (Implemented in Code)	5
2.5	Data Quality Checks	6
2.6	Length and Relevance Statistics	6
2.7	Category Distribution	6
2.8	Discussion of the Four EDA Graphs	7
2.9	Why This Section Matters for the Rest of the Project	8
3	Retrieval Methodology	8
3.1	TF-IDF Retrieval	8
3.2	BM25 Retrieval (Okapi and BM25+)	10
3.3	Embedding-based Retrieval	12
3.3.1	Encoding configurations evaluated	12
3.3.2	Encoding pipeline (implemented in code)	12
3.3.3	Vector structure and shape inspection	13
3.3.4	Similarity function and ranking	13
3.3.5	Experimental observations across k	13
3.3.6	2D projection (UMAP/t-SNE) and interpretation	14
4	Evaluation Framework	15
4.1	Evaluation Protocol	15
4.2	Metrics Used	15
4.3	Experimental Settings and Hyperparameters	16
4.4	Latency and Runtime Measurement	16
4.5	Stored Outputs	16
5	Experimental Results and Model Comparison	17
5.1	Quantitative Results	17
5.2	Comparative Analysis of Retrieval Performance	17
5.3	Effect of k on the Results	18
5.4	Latency and Trade-off Analysis	18
6	Kaggle Submission Strategy (Phase 1)	19
6.1	Selected Model for Phase 1 Submission	19
6.2	Submission Configuration	19
6.3	Submission Generation and Output	19
6.4	Why This Model Was Selected	20
7	Document & Query Classification	20
7.1	Model Choice and Methodology	20
7.2	Training Data Strategy	20
7.3	Feature Engineering and Ablation Study	21
7.4	Final Model Evaluation	22
7.5	Generalization: Docs vs Queries	23
8	Classification-Enhanced Retrieval	24
8.1	Updated Retrieval Pipeline	24
8.2	Updated Evaluation Metrics	25
8.3	Reranking Strategies	26
8.3.1	Hard Category Filtering	27
8.3.2	Soft Probabilistic Rescoring	27
8.3.3	Trade-offs and Expected Behavior	28
8.4	Experimental Results	28
9	Kaggle Submission Phase 2	30
9.1	Pipeline Configuration Phase 2	30
9.2	Results and Reproducibility	31
10	Conclusion	32

1 Introduction

1.1 Project Context

Information retrieval (IR) is a central problem in data science and machine learning: given a user query and a large document collection, the system must identify and rank the most relevant documents as early as possible. This ranking task is at the core of search engines, recommendation systems, and question-answering tools. In practice, effective retrieval requires combining complementary signals. Lexical methods are strong when query terms overlap directly with document terms, while semantic methods are needed when relevant content is expressed with different wording. Building a robust IR engine therefore means designing a pipeline that can balance exact matching, semantic understanding, and computational efficiency.

1.2 Objectives

The objective of this project is to design, implement, and evaluate an information retrieval system able to rank the most relevant documents for each query in a large StackExchange corpus spanning five technical domains. More broadly, the project aims to compare complementary retrieval strategies and to build a pipeline that is both effective and reproducible.

In Phase 1, the objective was to establish a strong retrieval baseline from raw data to competition-ready predictions. This included data preparation, construction of a unified `content` field from title, text, and tags, implementation of three retrieval families (TF-IDF, BM25/BM25+, and embeddings), and comparison under a unified evaluation protocol based on standard IR metrics and runtime analysis. The practical goal was to select the strongest configuration for the Phase 1 Kaggle submission.

Phase 2 extends this system by introducing supervised classification into the retrieval pipeline. The objective is to exploit the category structure of the dataset through query and document classification, compare several feature representations and classifiers, and integrate the resulting category signal into retrieval through category-aware reranking. To this end, we evaluate hard filtering and soft boosting, extend the evaluation framework with query category accuracy, and generate a final Kaggle submission based on the best end-to-end Phase 2 pipeline.

1.3 Scope and Positioning of This Report

This report covers the full Information Retrieval Engine project across both Phase 1 and Phase 2. It documents the complete methodological path of the system, from dataset understanding and preprocessing to retrieval modeling, evaluation, classification, reranking, and final submission generation.

The report is written from the implemented code and generated artifacts, ensuring that the reported decisions, results, and conclusions are reproducible. The analysis is grounded in both modular source code (`src/`) and notebook-based experimentation, so it reflects actual system behavior rather than theoretical expectations only.

More specifically, the document first presents the Phase 1 retrieval pipeline, which combines lexical and semantic methods under a unified evaluation framework. It then describes how Phase 2 extends this baseline with supervised classification, category-aware reranking, updated evaluation metrics, and a new Kaggle submission pipeline. In this sense, the report traces the evolution of the project toward a more integrated retrieval architecture.

1.4 Main Contributions

This project delivers a complete, modular, and reproducible information retrieval engine that addresses both phases of the assignment. Its main contributions can be summarized as follows:

Phase 1.

- A clean and reusable `src/` architecture separating data loading, preprocessing, retrieval, evaluation, and Kaggle submission logic.
- A robust content-construction pipeline that merges title, text, and tags into a unified `content` field for consistent downstream processing.
- The implementation and comparison of three retrieval families: TF-IDF, BM25/BM25+, and embedding-based retrieval with SentenceTransformers.
- A unified evaluation framework computing Precision@k, Recall@k, and MRR@k across multiple cutoffs, together with runtime measurements.
- The selection of a hybrid BM25+ + embeddings pipeline with Reciprocal Rank Fusion (RRF) as the strongest Phase 1 submission strategy.
- A fully automated Phase 1 submission workflow implemented in the `src/kaggle/` module, specifically in `submit_prep.py`.

Phase 2.

- The implementation of a document/query classification module satisfying the shared classification interface (`ClassifierProtocol`), enabling seamless integration with retrieval and evaluation.
- A complete ablation study over three feature families (TF-IDF, Count, embeddings) and four classifier families (Naïve Bayes, LinearSVC, Logistic Regression, MLP).
- The selection of TF-IDF + LinearSVC as the best classification model, achieving a validation macro-F1 of 0.9852.
- The integration of two category-aware reranking strategies, `hard_filter` and `soft_boost`, into the retrieval pipeline.
- An extension of the evaluation framework to include query category accuracy and joint retrieval–classification assessment.
- An end-to-end Phase 2 pipeline implemented in `src/kaggle/submit_phase2.py`, together with reproducible notebook-based experimentation for classification, pipeline evaluation, and submission.

Overall, the report contributes not only a working competition system, but also a reproducible experimental framework for analyzing how lexical retrieval, semantic retrieval, and supervised classification can be combined in a coherent information retrieval architecture.

2 Dataset Understanding

2.1 Data Sources and Schema

The project relies on four JSON files loaded through a unified utility, `load_all`, implemented in the `src/data/` module (`load.py`): `docs.json`, `queries_train.json`, `queries_test.json`, and `qgts_train.json`. This centralized loading step is important because it enforces a reproducible entry point for all notebooks and modules.

Documents and train queries share the same logical schema, with the fields `id`, `title`, `text`, `tags`, and `category`, while test queries are partially blind (no `category` label available at inference time). Ground-truth relevance is provided in `qgts_train.json` with a nested structure (`total_relevant_docs`, `relevant_doc_ids`, `category`). In later steps, this format is normalized by `adapt_ground_truth` (`src/evaluation/evaluate.py`) into a simple mapping `query_id → list[doc_id]` for consistent metric computation.

2.2 Global Dataset Scale

Indicator	Value
Number of documents	216,041
Number of train queries	327
Number of test queries	141
Ground-truth entries	327
Relevant query-document pairs	2,970

Table 1: Global size indicators of the dataset.

The ratio between documents and train queries is very high (about 660:1), which confirms that this is a large-candidate retrieval problem: for each query, only a tiny fraction of the corpus is relevant.

2.3 Field Availability and Practical Consequences

A direct inspection of raw files shows that:

- documents are almost fully populated for `title`, `text`, `tags`, `category`;
- train queries have `text`, `tags`, `category`, but no useful `title`;
- test queries have `text` and `tags`, but no `category`.

This has direct modeling consequences: the retrieval signal for queries is mostly carried by `text+tags`, and Phase 2 cannot rely on known test-query categories.

2.4 Content Construction and Preprocessing (Implemented in Code)

Retrieval is not performed on raw fields independently. Instead, the preprocessing module in `preprocess.py` constructs a unified `content` field, defined as `title + text + tags`, with robust handling of missing values.

The merge pipeline relies on three main functions:

- `to_string`: converts values to strings, handles `None`, trims spaces, and replaces the encoding artifact `\ufffd`;
- `build_content`: concatenates `title`, `text`, and `tags` in that order;
- `clean_text` (optional): applies lowercasing, whitespace normalization, and punctuation cleanup.

Listing 1 shows the implementation used to build `content` and sanitize noisy values before retrieval.

Listing 1: Robust construction of the `content` field

```
def to_string(v):
    """Convert a value to string, strip whitespace, and handle None."""
    if v is None:
        return ""
    return str(v).replace("\ufffd", " ").strip()

def build_content(item: dict) -> str:
    """
    Concatenate title, text, and tags while handling missing values.
    """
    parts = []

    title = to_string(item.get("title"))
    if title:
        parts.append(title)

    text = to_string(item.get("text"))
    if text:
        parts.append(text)

    tags = item.get("tags")
    if isinstance(tags, list):
        valid_tags = [to_string(t) for t in tags if to_string(t)]
        if valid_tags:
            parts.append(" ".join(valid_tags))
    else:
        tag_str = to_string(tags)
        if tag_str:
            parts.append(tag_str)

    return " ".join(parts)
```

In Notebook 01, descriptive statistics are computed with `clean=False`, while downstream retrieval outputs are exported with `clean=True` to stabilize tokenization.

2.5 Data Quality Checks

Notebook 01 includes an explicit quality-control block on generated `content` for documents, train queries, and test queries. It verifies missing keys, `None` values, empty strings, too-short text, and residual `\ufffd` characters. The final assertion reports **0 critical anomalies**, meaning the generated retrieval text is clean and usable.

One useful diagnostic result is that raw data contains 89 documents with `\ufffd` in `text` (about 0.04% of the corpus), but this corruption is neutralized by `to_string` and does not propagate into `content`. This is a small but important reproducibility detail: preprocessing is not only cosmetic, it prevents hidden text noise from affecting retrieval quality.

2.6 Length and Relevance Statistics

Using the generated `content` field (without cleaning for measurement), we obtain:

- document length: mean 137.2 words, median 100, std 147.0, min 9, max 5,256;
- query length: mean 11.4 words, median 11, std 3.9, min 3, max 31;
- relevant docs/query: mean 9.1, median 5, max 262.

These numbers show a strong asymmetry: queries are short and information-sparse, while documents are highly variable and often long. This is exactly the regime where pure lexical overlap can be insufficient and semantic matching becomes valuable.

2.7 Category Distribution

The corpus has five major categories with an imbalanced distribution. `tex` is the dominant class, followed by `unix` and `gaming`, while `android` is the smallest. Train-query categories are also imbalanced and do not perfectly mirror corpus proportions. This imbalance matters both for retrieval interpretation and for later classification/reranking in Phase 2.

2.8 Discussion of the Four EDA Graphs

Figure 1 summarizes four complementary aspects of the data.

(1) Document length distribution. The histogram is heavily right-skewed: most documents are short-to-medium length, but there is a very long tail. This pattern motivates length-aware lexical scoring (BM25 family), since raw term counts can over-favor long documents.

(2) Query length distribution. Query lengths are concentrated around 10–12 words, with limited variance and a few longer outliers. This confirms that user intent is expressed compactly; therefore small wording differences can produce large lexical mismatch.

(3) Top document categories. The bar chart confirms class imbalance, with `tex` clearly dominant. This suggests that category priors may influence ranking behavior and that category-aware reranking must be calibrated carefully to avoid bias.

(4) Relevant documents per query. This distribution is also long-tailed: many queries have relatively few relevant documents, while a few have very large relevant sets. As a consequence, evaluating only one cut-off k would be misleading. Low k emphasizes early precision (MRR/Hit@1), whereas higher k is required to assess recall on high-cardinality queries.

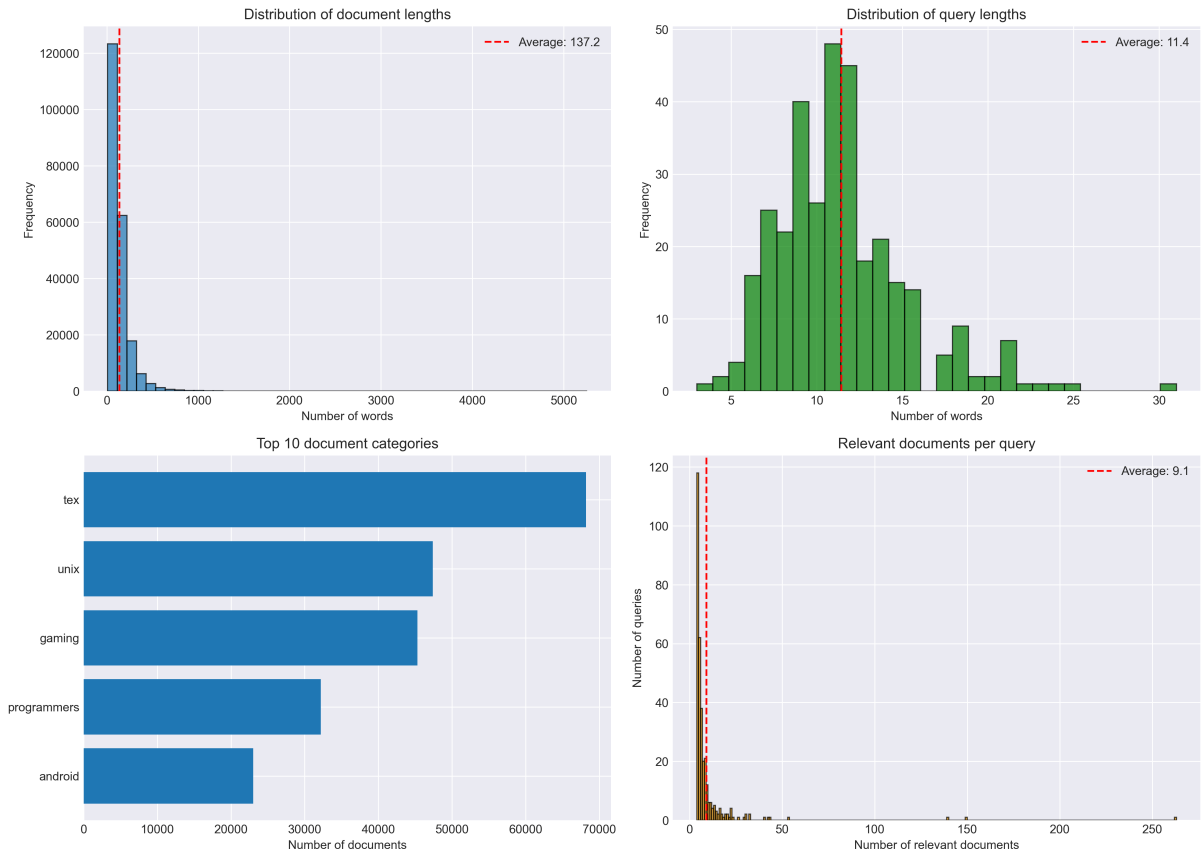


Figure 1: EDA overview: document lengths, query lengths, top document categories, and relevant documents per query.

2.9 Why This Section Matters for the Rest of the Project

This dataset analysis is not descriptive overhead; it directly drives modeling decisions. Length variability justifies BM25-style normalization. Short queries and sparse lexical evidence justify embedding-based retrieval. Class imbalance and missing test categories justify cautious use of category signals in Phase 2. Finally, the long-tail relevance distribution justifies reporting Precision@k, Recall@k, and MRR@k over multiple k values rather than a single score.

3 Retrieval Methodology

This section is the core of the project. It presents the full retrieval pipeline and the three model families implemented in our system: TF-IDF, BM25 (Okapi and BM25+), and embedding-based retrieval. The objective is to explain how each method represents text, computes relevance scores, and produces ranked top- k results for each query. Beyond implementation details, this section also highlights the rationale behind each approach, since these modeling choices directly determine the final retrieval quality, latency, and overall competition performance.

3.1 TF-IDF Retrieval

Principle. To begin with, we will explain TF-IDF. The Term Frequency–Inverse Document Frequency is a statistical method used in natural language processing and information retrieval to evaluate how important a word is to a document in relation to a larger collection of documents. It combines two components:

- **Term Frequency (TF):** that measures how often a word appears in a document.
- **Inverse Document Frequency (IDF):** that reduces the weight of common words across multiple documents while increasing the weight of rare words. An example of common words would be “the” or “of”.

The classic formulas are:

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t, D) = \log\left(\frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}\right)$$

For a term t in a document d :

$$TFIDF(t, d) = TF(t, d) \cdot \log\left(\frac{N}{df(t)} + 1\right)$$

where N is the total number of documents and $df(t)$ is the document frequency of t .

Implementation in our project. In our implementation, the documents’ `content` fields are fitted with the TF-IDF vectorizer from the Scikit-learn library.

As for the queries, they are transformed with the same fitter vectorizer in order to ensure consistent term weighting between documents and queries. We obtain relevance by computing cosine similarity between each query vector and the full document matrix:

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

where q is a query vector and d is a document-term matrix. The cosine similarity measures the cosine of the angle between two vectors. Cosine similarity ranges from -1 to 1, where 1 indicates identical texts, 0 means no similarity, and -1 denotes completely dissimilar texts.

Output contract (as required by the project statement). Finally, we get the top- k highest-scoring documents and their indices with our `retrieve_tfidf` function. We obtain `topk_indices` and `topk_scores`.

Chosen parameters and motivation.

- `sublinear_tf=True`: the TF component is calculated linearly. With this parameter, Scikit-Learn replaces the classic TF calculation with the following formula: $TF_{\log} = 1 + \log(TF)$. This logarithmic scale allows high frequencies to be “smoothed out” to prevent a term that is repeated too often from completely overwhelming the others in the calculation of the final score.
- `min_df=1`: this is the “Minimum Document Frequency” parameter. It defines the minimum number of documents in which a word must appear in order to be retained in the vocabulary. Here, it is initialized to 1 in order to avoid losing rare terms.
- `stop_words=None`: this concerns the common words. We set it to None in order to not filter any words. This can be useful for some queries.

Experimental observations. On our evaluation setup (`MAX_DOCS=5000`, train queries), TF-IDF reaches $\text{Precision@10} = 0.1997$, $\text{Recall@10} = 0.3247$, and $\text{MRR@10} = 0.4640$, with retrieval time around 0.085 s for the full query batch in Notebook 05. This confirms TF-IDF as a strong and efficient baseline, but not the best model in ranking quality.

Strengths and limitations. TF-IDF produces fast and sparse representations with low indexing and query times. This model provides a strong, interpretable lexical baseline. However, as a pure “bag of words” model, it has limitations in our context: It cannot capture semantic similarity like synonyms. It is sensitive to exact term matching and spelling variations. It struggles when relevant documents use different vocabulary from the query. These elements explain why our other models outperform TF-IDF on early precision and MRR metrics.

3.2 BM25 Retrieval (Okapi and BM25+)

Principle. BM25 is a probabilistic lexical ranking function used to estimate the relevance of a document D for a query Q . It is built on the same intuition as TF-IDF (term importance), but introduces two major improvements that are critical in practice:

- **Term-frequency saturation:** the contribution of a term increases quickly at first, then plateaus;
- **Document-length normalization:** long documents are not unfairly advantaged (or penalized) only because of size.

This usually makes BM25 more robust than TF-IDF on heterogeneous corpora with large length variance.

Okapi BM25 formulation. For query terms $q_i \in Q$, the Okapi BM25 score is:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

where:

- $f(q_i, D)$: frequency of term q_i in document D ,
- $|D|$: document length,
- avgdl : average document length in the corpus,
- k_1 : TF saturation control,
- b : length normalization control.

BM25+ formulation. BM25+ extends Okapi BM25 by adding a lower-bound term δ :

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \left[\frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} + \delta \right]$$

The δ term prevents excessive under-scoring in some long-document cases and can improve stability of lexical ranking depending on the corpus.

Implementation in our project. Our BM25 pipeline is implemented in the `src/retrieval/` module, in `bm25.py`, through the function `rank_bm25`. We evaluate both variants:

- `BM25Okapi (method="okapi")`,
- `BM25Plus (method="plus")`.

A specific design choice in our BM25 implementation involves the tokenization process. Standard tokenizers often strip out non-alphabetic characters, which is detrimental for technical domains where terms like `c++`, `node.js`, or `.net` carry critical semantic meaning.

To preserve these technical tokens, we implemented a custom tokenizer (`re.findall(r'[a-zA-Z0-9.+]+'`, `text`)) in our preprocessing pipeline (`bm25.py`). This simple yet highly effective approach converts the text to lowercase and explicitly extracts fragments containing alphanumeric characters alongside periods (`.`) and plus signs (`+`). As a result, technical entities are kept intact rather than being split into meaningless sub-tokens (e.g., `node` and `js`), significantly improving the lexical matching capability on our specific Kaggle corpus.

Listing 2 shows the core retrieval logic.

Listing 2: Core BM25 retrieval pipeline

```
tokenized_texts = [tokenize(doc["content"]) for doc in docs]

if method == "plus":
    bm25_model = BM25Plus(tokenized_texts)
else:
    bm25_model = BM25Okapi(tokenized_texts)

tokenized_query = tokenize(query["content"])
scores = np.array(bm25_model.get_scores(tokenized_query))
topk = np.argsort(scores)[-1:-k:-1]
```

Output contract (project requirement). The BM25 retriever returns:

$$\text{topk_indices_bm25} \in \mathbb{N}^{n_q \times k}, \quad \text{topk_scores_bm25} \in \mathbb{R}^{n_q \times k},$$

which matches the required top- k output format for Phase 1.

Experimental observations. On our common evaluation setup (`MAX_DOCS=5000`, train queries), both BM25 variants are very close at $k = 10$:

- **BM25 Okapi:** Precision@10 = 0.2526, Recall@10 = 0.4092, MRR@10 = 0.6288;
- **BM25+:** Precision@10 = 0.2532, Recall@10 = 0.4071, MRR@10 = 0.6250.

BM25+ is slightly better on Precision@10, while Okapi is slightly better on Recall@10 and MRR@10 in this specific run. Overall, both are strong lexical baselines and clearly improve over TF-IDF.

Strengths and limitations. BM25 is robust, interpretable, and generally stronger than TF-IDF when document lengths vary. However, it remains lexical: it still depends on term overlap. For example, a query like *“fix login credentials”* may miss a relevant document phrased as *“reset account password”* if shared vocabulary is limited. This lexical limitation motivates embedding-based retrieval in the next subsection.

3.3 Embedding-based Retrieval

Why embeddings for retrieval. Lexical models such as TF-IDF and BM25 rely on exact or near-exact token overlap between query and document. This creates a structural limitation: if a query uses one wording and a relevant document uses a semantic paraphrase, lexical scoring can remain low despite real relevance. For instance, a query such as “*how to fix a broken pipe*” and a document discussing “*plumbing repair*” may share little lexical overlap while being semantically equivalent. Embedding-based retrieval addresses this issue by mapping documents and queries into a shared dense vector space, where geometric proximity reflects semantic relatedness rather than exact word matching.

Model choice. Our main embedding model is `all-MiniLM-L12-v2` (SentenceTransformers), selected for its strong quality/speed trade-off. It outputs 384-dimensional vectors and is significantly lighter than larger models (e.g., MPNet-based variants), which is important for iterative experimentation and Kaggle-style runtime constraints.

3.3.1 Encoding configurations evaluated

We evaluated three practical encoding presets all with `all-MiniLM-L12-v2` model:

Config	Batch size	Precision	Max seq. length	Truncation dim
L12quality	64	float32	None	None
L12balanced128	128	float32	128 tokens	384
L12fast96	256	int8	96 tokens	256

Table 2: Embedding configurations tested in the notebook experiments.

All configurations use `normalize_embeddings=True`, so vectors are unit-normalized.

3.3.2 Encoding pipeline (implemented in code)

The embedding workflow (see `src/retrieval/embeddings.py`) is:

1. Build input text from the unified `content` field (`title + text + tags`).
2. Sanitize null values (fallback to empty string).
3. Encode documents and queries with SentenceTransformers in batches.
4. Produce dense vectors of dimension d (typically $d = 384$).
5. Cache vectors to `.npy` files using SHA-256 fingerprints of input texts.

This cache strategy is crucial: encoding is the expensive stage, while retrieval itself is extremely fast once vectors are available.

3.3.3 Vector structure and shape inspection

In the run shown in the notebook, the printed shapes are:

```
doc_emb shape = (5000, 384),    query_emb shape = (46, 384).
```

Interpretation:

- 5000 rows = one embedding per document in the capped subset (MAX_DOCS=5000);
- 46 rows = queries retained for this subset evaluation;
- 384 columns = embedding dimension of `all-MiniLM-L12-v2`.

3.3.4 Similarity function and ranking

Given query vector $q \in \mathbb{R}^d$ and document vector $d_i \in \mathbb{R}^d$:

$$\text{sim}(q, d_i) = \frac{q \cdot d_i}{\|q\| \|d_i\|}.$$

Because vectors are L2-normalized, this simplifies to:

$$\text{sim}(q, d_i) = q \cdot d_i.$$

The full query-document similarity matrix is computed efficiently by matrix multiplication. Top- k retrieval is then obtained via `np.argpartition` (fast candidate selection) followed by local re-sorting for exact rank order.

As required by the project statement, the retriever returns:

$$\text{topk_indices} \in \mathbb{N}^{n_q \times k}, \quad \text{topk_scores} \in \mathbb{R}^{n_q \times k}.$$

3.3.5 Experimental observations across k

On the reported run (subset setting), the embedding retriever yields:

k	retrieval_time_s	precision@k	recall@k	mrr@k	phase1_score
5	0.008070	0.121739	0.467391	0.381159	0.323430
10	0.003223	0.076087	0.590062	0.394048	0.353399
20	0.002213	0.046739	0.645963	0.396856	0.363186
50	0.002205	0.021304	0.732402	0.399863	0.384523

Table 3: Embedding retrieval performance across different k .

These results show the classical trade-off: precision decreases as k increases, while recall increases. MRR improves then stabilizes, indicating that first relevant documents are already found relatively early. Retrieval latency is in the millisecond range after embeddings are built.

3.3.6 2D projection (UMAP/t-SNE) and interpretation

To inspect semantic structure, embeddings are projected to 2D (UMAP preferred; t-SNE fall-back). In our setup, UMAP uses `n_neighbors=12` and `min_dist=0.15`, giving a good balance between local cohesion and global readability. The projection (Figure 2) shows:

- visible category-level clusters (semantic coherence),
- partial overlap between related technical domains,
- query points generally placed near plausible document regions.

This is expected: domains such as `programmers`, `unix`, and `android` share vocabulary and concepts, so boundaries are not perfectly separable. Also, any 2D projection of a 384D space introduces distortion, so overlap should be interpreted cautiously.

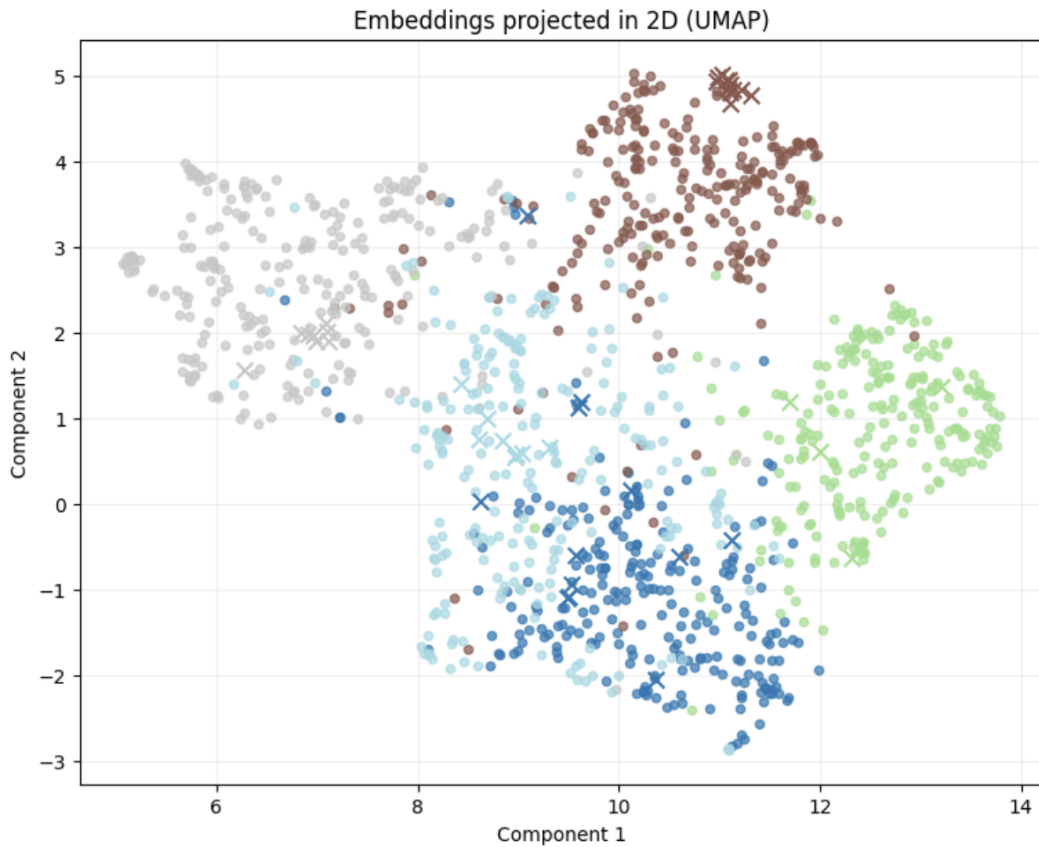


Figure 2: 2D UMAP projection of document and query embeddings.

Strengths and limitations. Embedding-based retrieval is robust to synonymy and paraphrase, and it provides the best semantic recall behavior in our experiments. Its main cost is the initial encoding stage (heavier than lexical indexing), but this is largely mitigated by embedding caching. Overall, this method is the strongest semantic component of the pipeline and a key contributor to final ranking quality.

4 Evaluation Framework

4.1 Evaluation Protocol

The evaluation is performed on `queries_train.json` using the official relevance judgments from `qgts_train.json`. To ensure reproducibility and fair comparison across models, the protocol is centralized in `src/evaluation/evaluate.py` and `src/evaluation/metrics.py`.

For each run (TF-IDF, BM25 Okapi/BM25+, embeddings, hybrid), we apply the same sequence:

1. adapt raw ground-truth format with `adapt_ground_truth()`,
2. convert retrieved indices to document IDs with `indices_to_docids()`,
3. compute retrieval metrics with `evaluate_run()`.

This shared pipeline avoids model-specific evaluation bias and guarantees that score differences come from retrieval quality, not from metric implementation differences.

4.2 Metrics Used

We report three standard IR metrics at multiple cut-offs k : Precision@ k , Recall@ k , and MRR@ k .

Let $Rel(q)$ be the relevant set for query q , and $Top_k(q)$ the top- k retrieved results.

$$\text{Precision@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|Rel(q) \cap Top_k(q)|}{k}$$

Precision measures *result purity*: among retrieved results, how many are relevant.

$$\text{Recall@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|Rel(q) \cap Top_k(q)|}{|Rel(q)|}$$

Recall measures *coverage*: among all relevant documents, how many are found.

$$\text{MRR@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}$$

where rank_q is the rank of the first relevant result in $Top_k(q)$, and 0 if none appears.

In practice, Precision@ k and Recall@ k characterize different trade-offs, while MRR@ k emphasizes early ranking quality (how quickly the first relevant document appears).

4.3 Experimental Settings and Hyperparameters

The unified benchmark is run from `notebooks/05_evaluation_suite.ipynb` with common settings:

- evaluated cut-offs: $k \in \{1, 5, 10, 20, 50\}$,
- benchmark subset size: `MAX_DOCS=5000`,
- shared text input for all models: `content (title + text + tags)`,
- TF-IDF parameters: `sublinear_tf=True, min_df=1, stop_words=None`,
- BM25 variants: `okapi` and `plus`,
- embedding models compared in this suite: `all-MiniLM-L6-v2`, `all-MiniLM-L12-v2`, and `multi-qa-mpnet-base-dot-v1`.

The candidate subset is deterministic and includes relevant documents first, then additional corpus documents up to `MAX_DOCS`, preserving evaluation validity while keeping runtime tractable.

4.4 Latency and Runtime Measurement

Runtime is measured with `time.perf_counter()` and split into:

- `fit_time_s`: index/representation build time,
- `retrieval_time_s`: time to score and rank a full query batch for a given k .

This distinction is important: some models are expensive to build but fast at query time (e.g., embeddings with cached vectors), while others have different fit/retrieval trade-offs.

4.5 Stored Outputs

Evaluation artifacts are exported to:

- `outputs/runs/evaluation_suite_results.csv`,
- `outputs/runs/evaluation_suite_best_by_k.csv`,
- `outputs/runs/evaluation_suite_best_for_top1.csv`.

The main table includes model identity, k , Precision@ k , Recall@ k , MRR@ k , and timing columns, providing a single consistent basis for model comparison.

5 Experimental Results and Model Comparison

5.1 Quantitative Results

This section reports the empirical results obtained on the training split with the unified evaluation suite (`notebooks/05_evaluation_suite.ipynb`), using the common setup introduced in Section 4 (same preprocessing, same query set, same metrics, and comparable candidate pool).

Table 4 summarizes model performance at $k = 10$, which is a useful operating point to compare ranking quality and coverage simultaneously.

Model	Precision@10	Recall@10	MRR@10	Fit (s)	Retr. (s)
TF-IDF	0.1997	0.3247	0.4640	0.3991	0.0853
BM25 Okapi	0.2526	0.4092	0.6288	0.2632	2.2240
BM25+	0.2532	0.4071	0.6250	0.8394	2.3864
all-MiniLM-L6-v2	0.3190	0.5212	0.7022	45.3261	0.0142
all-MiniLM-L12-v2	0.3547	0.5731	0.7458	12.5981	0.0143
multi-qa-mpnet-base-dot-v1	0.3092	0.5029	0.6680	254.4883	0.0159

Table 4: Comparison of retrieval models at $k = 10$.

The main quantitative observations are:

- Embedding models clearly outperform lexical models on all quality metrics.
- **all-MiniLM-L12-v2** is the best model in this benchmark (best Precision@10, Recall@10, and MRR@10).
- BM25 variants are consistently stronger than TF-IDF on ranking quality.
- BM25 Okapi and BM25+ are very close; BM25+ has a slight precision edge, while Okapi has slightly better Recall/MRR in this run.

5.2 Comparative Analysis of Retrieval Performance

The results confirm a clear hierarchy between model families. TF-IDF provides a solid and interpretable baseline, but its lexical nature limits its ability to retrieve semantically relevant documents when query wording and document wording differ. BM25 improves this lexical baseline through TF saturation and document-length normalization, which explains its consistent gains over TF-IDF.

Embedding-based retrieval produces the best overall ranking quality because it encodes semantic proximity rather than exact token overlap. This is especially visible in MRR, where **all-MiniLM-L12-v2** strongly dominates lexical methods, meaning relevant documents are found earlier in ranked lists.

A key nuance is that not all embedding models are equal in practice: **all-MiniLM-L12-v2** gives the best quality-speed balance in our experiments, outperforming **all-MiniLM-L6-v2** and being much cheaper to build than **multi-qa-mpnet-base-dot-v1**.

5.3 Effect of k on the Results

Across all models, we observe the expected IR behavior:

- Precision decreases as k increases,
- Recall increases as k increases,
- MRR improves quickly at low k and then tends to stabilize.

For the best model (**all-MiniLM-L12-v2**), this trend is explicit:

k	Precision@ k	Recall@ k	MRR@ k	retrieval_time_s
5	0.5113	0.4435	0.7390	0.0148
10	0.3547	0.5731	0.7458	0.0143
20	0.2283	0.6850	0.7480	0.0145
50	0.1168	0.7951	0.7487	0.0151

Table 5: Performance trend of **all-MiniLM-L12-v2** across k .

This indicates that early relevant hits are already strong at small k , while larger k mainly improves coverage.

5.4 Latency and Trade-off Analysis

Runtime analysis highlights two distinct costs:

- **Fit/build cost** (indexing/representation construction),
- **Retrieval cost** (query-time ranking).

TF-IDF and BM25 have low fit costs (sub-second in this setup), while embeddings require a much larger one-time build cost. However, once vectors are built (and cached), embedding retrieval is extremely fast (milliseconds), while BM25 retrieval remains much slower in this notebook setup (around 2.2–2.4 seconds per full query batch).

Therefore, the practical trade-off is:

- TF-IDF: fast baseline, limited ranking quality;
- BM25: stronger lexical quality, slower retrieval than TF-IDF;
- Embeddings: highest quality and very fast retrieval after an expensive build step.

Overall, **all-MiniLM-L12-v2** offers the best quality/performance compromise in our experimental conditions.

6 Kaggle Submission Strategy (Phase 1)

6.1 Selected Model for Phase 1 Submission

For Phase 1, the selected submission pipeline is the hybrid BM25+ + embeddings retriever:

```
RETRIEVAL_METHOD = "hybrid_bm25_embeddings".
```

This method combines lexical and semantic signals through weighted Reciprocal Rank Fusion (RRF), and corresponds to the configuration validated in `src/config.py` with `top_k=100` and `category="?"`.

6.2 Submission Configuration

The final Phase 1 configuration is summarized in Table 6.

Component	Parameter	Value
Retrieval method	<code>retrieval_method</code>	<code>hybrid_bm25_embeddings</code>
Embedding model	<code>embedding_model_name</code>	<code>all-MiniLM-L12-v2</code>
BM25 variant	<code>hybrid_bm25_method</code>	<code>plus</code>
RRF constant	<code>hybrid_rrf_k</code>	<code>20</code>
Embedding weight	<code>hybrid_weight_embeddings</code>	<code>3.5</code>
BM25 weight	<code>hybrid_weight_bm25</code>	<code>0.5</code>
Submission cutoff	<code>top_k</code>	<code>100</code>
Category placeholder	<code>category_value</code>	<code>"?"</code>

Table 6: Final hybrid submission configuration for Kaggle Phase 1.

For a document d , with embedding rank $r_e(d)$ and BM25 rank $r_b(d)$, the fused score is:

$$\text{RRF}(d) = \frac{w_e}{k_{\text{rrf}} + r_e(d)} + \frac{w_b}{k_{\text{rrf}} + r_b(d)},$$

with $w_e = 3.5$, $w_b = 0.5$, and $k_{\text{rrf}} = 20$. This weighting deliberately gives more importance to semantic retrieval while preserving a lexical correction signal for exact technical terms.

6.3 Submission Generation and Output

The end-to-end export is implemented in `src/kaggle/submit_prep.py`. The pipeline loads the test queries, retrieves the top-100 documents with the hybrid model, converts indices to document IDs, and writes the final CSV in Kaggle format.

In the reported full-corpus run, the output contains:

- 141 test queries,
- a top-100 ranking for each query,
- a valid CSV file with columns `query_id`, `relevant_doc_ids`, and `category`.

The generated file `outputs/submissions/submission.csv` passes all structural checks, including JSON validity of the ranked document list and the placeholder category value `"?"`.

6.4 Why This Model Was Selected

The hybrid submission was retained because it provides the best compromise between lexical precision and semantic robustness. Pure lexical systems remain effective on exact terminology, while dense embeddings are better at handling paraphrases and semantic reformulations. Their fusion reduces failure cases and produces a more stable ranking across heterogeneous query types.

Since Kaggle submission is an offline batch process, the additional runtime cost of the hybrid model remains acceptable. Reproducibility is ensured by the deterministic retrieval pipeline and by the programmatic validation of the final file format before export.

7 Document & Query Classification

Phase 2 adds a category-prediction module that assigns one of five labels (`android`, `gaming`, `programmers`, `tex`, `unix`) to documents and queries, then uses this signal for reranking.

7.1 Model Choice and Methodology

The classification component follows the shared interface defined in `interfaces.py`, which exposes `fit`, `predict`, `predict_proba`, and `classes_`. Its concrete implementation is the `Classifier` wrapper in `model.py`, which standardizes four candidate models: Multinomial Naïve Bayes, LinearSVC, Logistic Regression, and MLP.

This wrapper makes the classification stage modular and compatible with the Phase 2 pipeline. All models are trained through the same API with `RANDOM_SEED = 42`. When supported, `predict_proba` returns class probabilities; for `LinearSVC`, it returns `None`.

7.2 Training Data Strategy

The classifier is trained on both the document collection and the labeled training queries. In Notebook 07, the data are assembled with `extract_texts_and_labels(docs, queries_train)`, which merges the `content` field and the corresponding category labels from both sources.

Source	Samples	Role
Documents	216,041	Main category-specific vocabulary source
Train queries	327	Short query-style supervision
Total	216,368	Unified labeled corpus

Table 7: Data used for Phase 2 classification.

This mixed strategy improves generalization: documents provide most of the supervision, while `queries_train` expose the model to short texts closer to the final inference setting. The dataset is then split with a stratified 70%/15%/15% train/validation/test split.

Listing 3: Training data assembly and split

```
texts, labels = extract_texts_and_labels(docs, queries_train)

X_train, y_train, X_val, y_val, X_test, y_test = stratified_split(
    texts, labels, val_size=0.15, test_size=0.15, random_state=RANDOM_SEED
)
```

7.3 Feature Engineering and Ablation Study

Three feature families are implemented in `src/classification/features.py`: TF-IDF, Count Vectorizer, and dense embeddings. Table 8 summarizes their main configurations.

Feature family	Implementation	Main parameters
TF-IDF	<code>TfidfVectorizer</code>	<code>max_features=50,000</code> , <code>ngram_range=(1,2)</code> , <code>sublinear_tf=True</code>
Count	<code>CountVectorizer</code>	<code>max_features=50,000</code> , <code>ngram_range=(1,2)</code>
Embedding	<code>SentenceTransformers</code>	<code>all-MiniLM-L12-v2</code> , batch size 64

Table 8: Main feature configurations used in the classification ablation.

The final feature configuration is a TF-IDF representation with `max_features=50_000`, `ngram_range=(1,2)`, and `sublinear_tf=True`.

The notebook evaluates all combinations of 3 feature sets and 4 classifiers. In practice, 11 combinations are tested because Multinomial Naïve Bayes is not used with embeddings. Methodologically, the ablation is performed **only on the validation set**; the held-out test set is kept untouched and is used only after the best model has been selected.

Listing 4: Feature construction used in the ablation study

```
tfidf_vec, X_train_tfidf, X_val_tfidf, X_test_tfidf = \
    build_tfidf_features(X_train, X_val, X_test)

count_vec, X_train_count, X_val_count, X_test_count = \
    build_count_features(X_train, X_val, X_test)

_, X_train_emb, X_val_emb, X_test_emb = \
    build_embedding_features(X_train, X_val, X_test)
```

The validation results are reported in Table 9.

Features	Classifier	Accuracy	Macro-F1	Bal. Acc.
tfidf	svc	0.986998	0.985151	0.984817
tfidf	logreg	0.981883	0.979433	0.978597
count	svc	0.981760	0.979339	0.978796
tfidf	mlp	0.980866	0.978453	0.977934
count	logreg	0.980004	0.977644	0.976992
count	mlp	0.978895	0.976137	0.975503
embedding	mlp	0.972270	0.969461	0.968628
tfidf	nb	0.964321	0.961863	0.962983
count	nb	0.957881	0.953905	0.960931
embedding	svc	0.945804	0.944776	0.943427
embedding	logreg	0.944201	0.943494	0.941963

Table 9: Validation-set ablation study for Phase 2 classification.

Two conclusions emerge clearly. First, TF-IDF is the best feature family on this corpus, which indicates that category discrimination relies strongly on lexical cues. Second, LinearSVC is the best classifier on sparse features. The final choice is therefore **TF-IDF + LinearSVC**, with macro-F1 = 0.9852, accuracy = 0.9870, and balanced accuracy = 0.9848 on the validation set. A practical limitation is that LinearSVC does not provide class probabilities: if a probabilistic reranking strategy is needed, LogisticRegression is the natural alternative, which explains why it will remain relevant in Section 8.

7.4 Final Model Evaluation

After selecting the best configuration on the validation set, the model is retrained on the union of the training and validation sets, then evaluated on the held-out test set.

Listing 5: Final training and evaluation of the selected classifier

```
y_trainval = y_train + y_val
X_trainval_tfidf = vstack([X_train_tfidf, X_val_tfidf])

clf = Classifier(method="svc")
clf.fit(X=X_trainval_tfidf, y=y_trainval)

y_pred = clf.predict(X_test_tfidf)
```

Accuracy is computed as the fraction of correctly classified samples:

$$\text{Accuracy} = \frac{\text{correctly classified samples}}{\text{total number of samples}}.$$

On the held-out test set (32,456 samples), the final classifier reaches:

Accuracy = 0.9876, Macro-F1 = 0.9855, Balanced Accuracy = 0.9852.

Category	Precision	Recall	F1-score	Support
android	0.98	0.98	0.98	3,458
gaming	0.99	0.99	0.99	6,802
programmers	0.98	0.98	0.98	4,834
tex	1.00	1.00	1.00	10,247
unix	0.98	0.98	0.98	7,115
Accuracy		0.9876		32,456
Macro average	0.99	0.99	0.99	32,456
Weighted average	0.99	0.99	0.99	32,456

Table 10: Held-out test-set classification report for the final model.

The results are excellent across all five classes. **tex** is nearly perfect, which is expected given its highly distinctive vocabulary. The rare confusions mostly involve **android**, **programmers**, and **unix**, three categories that share overlapping technical terms. This pattern is confirmed by the confusion matrix shown in Figure 3.

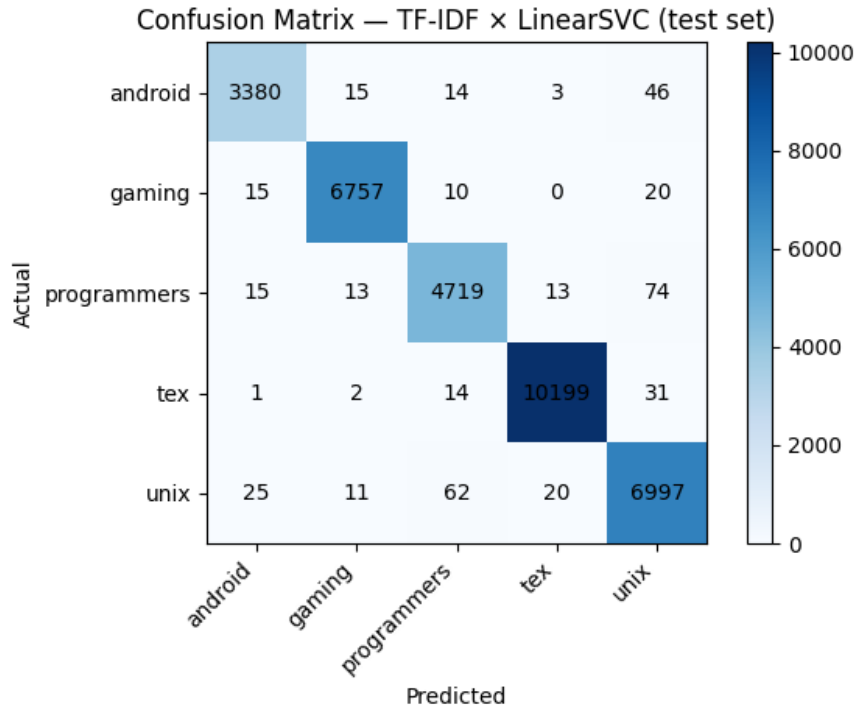


Figure 3: Confusion matrix of the final TF-IDF + LinearSVC classifier on the held-out test set. Rows correspond to ground-truth labels and columns to predicted labels. Most errors are concentrated between **android**, **programmers**, and **unix**, which share overlapping technical vocabulary.

The matrix shows that most remaining errors are concentrated between **programmers** and **unix**, and to a lesser extent between **android** and **unix**. These confusions are semantically plausible, since shell commands, package names, and general development vocabulary are shared across these categories.

7.5 Generalization: Docs vs Queries

Since the classifier is mainly trained on documents but applied to queries at inference time, Notebook 07 also evaluates performance separately on the full document collection and on `queries_train`. This comparison is implemented with `evaluate_docs_vs_queries(...)`.

Split	Accuracy	Macro-F1	Bal. Acc.	Samples
Documents	1.00	1.00	1.00	216,041
Queries	0.99	0.99	0.99	327

Table 11: Generalization of the final classifier from documents to queries.

The performance gap is negligible, which shows that the classifier generalizes very well from long documents to short queries. On queries, the only slightly weaker categories are **android** (recall = 0.96) and **unix** (precision = 0.96), which is consistent with the overlap between mobile development, general programming, and shell-oriented questions.

Overall, the classification component is accurate and robust enough to be used as a reliable signal for Phase 2 retrieval enhancement.

8 Classification-Enhanced Retrieval

8.1 Updated Retrieval Pipeline

Phase 2 extends the retrieval pipeline by integrating query category prediction directly into the end-to-end inference process. The goal is no longer to return only a ranked list of document IDs, but also to predict a category for each query and use this signal to guide reranking before submission generation.

In the implemented pipeline (`src/kaggle/submit_phase2.py`), the process begins by loading the raw Phase 2 files and rebuilding the `content` field for documents, training queries, and test queries. The classifier is then trained on the combined set of documents and labeled training queries, using the feature family and classifier family specified in `Phase2Config`. In the final configuration, this corresponds to a TF-IDF representation and a linear SVM classifier.

Once trained, the classifier predicts one category for each test query. These predicted labels are not treated as a separate output only: they are injected back into the retrieval process in two successive ways. First, each query is expanded with a small category-specific vocabulary, so that lexical and semantic retrieval are conditioned by the predicted domain. Second, after first-stage retrieval, the same predicted category is passed to the reranking module, which applies either `hard_filter` or `soft_boost` depending on the selected strategy.

The implemented Phase 2 pipeline therefore follows these steps:

1. load raw documents, training queries, and test queries;
2. rebuild the `content` field with the same preprocessing logic as in the notebooks;
3. train the classifier on documents + `queries_train`;
4. predict the category of each test query;
5. expand each query with category-specific vocabulary;
6. apply pseudo-relevance feedback (PRF) by retrieving top BM25 candidates and appending frequent tags to the query;
7. run hybrid BM25 + embeddings retrieval with Reciprocal Rank Fusion;
8. rerank the retrieved candidates using the predicted category;
9. export the final submission with both document rankings and predicted query categories.

This behavior is implemented in the end-to-end function `run_phase2_pipeline()`. Internally, the pipeline builds a dictionary mapping each query ID to its predicted category, and this mapping is passed to `save_submission(..., categories=...)` so that the final output file contains both the ranked document list and the predicted class for each query.

Listing 6: Core Phase 2 pipeline: category prediction integrated into retrieval

```

predicted_categories = clf.predict(X_test)

queries_expanded = []
for q, cat in zip(queries_test, predicted_categories):
    q_exp = q.copy()
    q_exp["content"] = q["content"] + " " + _CATEGORY_EXPANSIONS.get(cat, cat)
    queries_expanded.append(q_exp)

topk_indices, topk_scores = retrieve_hybrid_bm25_embeddings(
    docs=docs,
    queries=queries_final,
    top_k=retrieval_k,
    text_field=config.text_field,
    cache_dir=paths.cache_dir,
    embedding_model_name=config.embedding_model_name,
    hybrid_weight_embeddings=config.hybrid_weight_embeddings,
    hybrid_weight_bm25=config.hybrid_weight_bm25,
    hybrid_rrf_k=config.hybrid_rrf_k,
)

```

Compared with the Phase 1 pipeline, the key conceptual change is that classification is no longer external to retrieval. It becomes an intermediate prediction layer that influences both query formulation and final ranking, while preserving the same output contract required for Kaggle submission: one row per query, a top-100 ranked list of document IDs, and a predicted category.

8.2 Updated Evaluation Metrics

Phase 2 extends the evaluation framework so that retrieval quality and classification quality can be assessed jointly. The Phase 1 metrics remain unchanged: Precision@k, Recall@k, and MRR@k are still used to evaluate how well the system ranks relevant documents. However, because the pipeline now predicts a category for each query and uses this prediction during reranking, classification performance must also be measured explicitly.

To support this new setting, the project implements a set of classification-oriented metrics in `src/evaluation/metrics.py`. These include:

- `accuracy`, which measures the overall fraction of correctly classified samples;
- `macro_f1`, which computes the macro-averaged F1 score across the five categories;
- `balanced_accuracy`, which averages recall across classes and is therefore less sensitive to class imbalance;
- `classification_report_phase2`, which returns the full per-class precision, recall, and F1 summary;
- `confusion_matrix_phase2`, which returns the corresponding confusion matrix.

At pipeline level, the key additional metric is `query_category_accuracy`. This metric measures the fraction of queries for which the predicted category matches the ground-truth category. It is the most directly relevant measure for Phase 2, because reranking decisions are driven by the predicted query label.

Formally, if Q is the set of evaluated queries, the metric is defined as:

$$\text{Query Category Accuracy} = \frac{\text{number of correctly classified queries}}{\text{total number of evaluated queries}}.$$

In the code, this metric takes as input the ordered query IDs, the predicted category of each query, and the corresponding ground-truth categories. Queries missing from the ground truth are ignored with a warning, while missing predictions raise an explicit error.

The joint Phase 2 evaluation helper is implemented in `evaluate.py` through `evaluate_with_classification` which combines the standard retrieval metrics from `evaluate_run(...)` with `query_category_accuracy`. The output is a single dictionary containing both retrieval and classification results.

Listing 7: Joint Phase 2 evaluation: retrieval + category prediction

```
def evaluate_with_classification(
    pred_docids,
    gt,
    query_ids,
    k,
    predicted_categories,
    gt_categories,
):
    retrieval_metrics = evaluate_run(pred_docids, gt, query_ids, k)
    category_accuracy = query_category_accuracy(
        query_ids, predicted_categories, gt_categories
    )
    return {**retrieval_metrics, "query_category_accuracy": category_accuracy}
```

This update is important methodologically. In Phase 1, a system could be evaluated only by the quality of its ranking. In Phase 2, this is no longer sufficient, because the predicted query category is itself a model output and directly influences the reranking stage. Reporting both retrieval metrics and query category accuracy therefore provides a more complete and faithful evaluation of the end-to-end system.

8.3 Reranking Strategies

The classifier trained in Phase 2 introduces a new source of information that can be exploited to improve retrieval quality. If a query is predicted to belong to a given category, then documents from the same category are, on average, more likely to be relevant than documents from unrelated categories. Since test queries are not labeled, this predicted category becomes the only available category signal at inference time.

In this setting, reranking is a post-retrieval step. A first-stage retriever produces a candidate list of documents and scores, and a second stage modifies this ranking using the classifier output. In the project, two complementary reranking strategies are implemented in `src/classification/rerank.py`: a hard category-based reordering and a soft probabilistic score adjustment.

8.3.1 Hard Category Filtering

The first strategy, implemented by `hard_filter(...)`, uses the single category predicted for the query. The candidate list is split into matching and non-matching documents: matching ones are promoted to the top of the ranking, while the others are moved to the end.

With `fallback_to_original=True`, no document is removed. The method preserves the full candidate list and only reorders it, so the downstream output contract remains unchanged.

Listing 8: Core logic of hard category filtering

```
if doc["category"] == predicted_category:
    matched_ids.append(doc_id)
    matched_scores.append(score)
else:
    excluded_ids.append(doc_id)
    excluded_scores.append(score)

if fallback_to_original and len(matched_ids) < len(topk_doc_ids):
    matched_ids += excluded_ids
    matched_scores += excluded_scores
```

This behavior is deliberate. A strict filtering strategy that discards all non-matching documents would be too aggressive in the presence of classifier uncertainty, especially for queries near category boundaries. In practice, `hard_filter` should therefore be interpreted as a strong categorical prior: it promotes category-consistent candidates, but preserves the original fallback list in case relevant cross-category documents exist.

8.3.2 Soft Probabilistic Rescoring

The second strategy, implemented by `soft_boost(...)`, uses the full probability distribution returned by the classifier instead of only the top predicted label. Each candidate document receives a multiplicative score boost based on the predicted probability of its category:

$$\text{score}_{\text{boosted}} = \text{score}_{\text{retrieval}} \cdot (1 + \text{boost_factor} \cdot p(\text{doc category} \mid \text{query})).$$

The parameter `boost_factor` controls the strength of the category signal. After rescoring, the documents are sorted again in decreasing order of boosted score.

Listing 9: Soft probabilistic rescoring used in Phase 2

```
proba = prob_by_cat.get(doc_category, 0.0)
new_score = score * (1 + boost_factor * proba)
boosted_scores.append(new_score)

sorted_indices = np.argsort(boosted_scores)[::-1]
reranked_doc_ids = [topk_doc_ids[i] for i in sorted_indices]
```

This method is more flexible than hard filtering because it does not commit to a single category. Documents from secondary plausible categories may still receive a partial boost, which makes the method more robust to classification uncertainty. The implementation also degrades gracefully: if `category_proba` is `None`, the original ranking is returned unchanged.

8.3.3 Trade-offs and Expected Behavior

Both reranking strategies operate on the output of the first-stage retriever, that is, on candidate document IDs and their scores. The choice between **hard_filter** and **soft_boost** reflects how strongly the classifier signal should influence the final ranking.

hard_filter is the stronger strategy and is best suited to high-confidence predictions with sharp category boundaries. In our corpus, however, category structure is informative but not absolute, which makes the fallback mechanism essential. **soft_boost** is more conservative: it adjusts scores rather than restructuring the ranking, and is therefore better suited to uncertain or mixed-category cases.

In both cases, the goal is to improve early-ranking metrics such as Precision@k and MRR@k by surfacing category-consistent documents earlier, although too strong a category bias may reduce Recall@k. The next subsection quantifies this trade-off.

8.4 Experimental Results

All experiments in this section are conducted on the full corpus of 216,041 documents and the 327 training queries, so that evaluation remains comparable to the final competition setting. As shown in Notebook 08, the classifier used for reranking is TF-IDF + Logistic Regression rather than the final TF-IDF + LinearSVC model from Section 7, because **soft_boost** requires class probabilities. On the validation split, Logistic Regression reaches Accuracy = 0.9819, Macro-F1 = 0.9794, and Balanced Accuracy = 0.9786, while achieving a query category accuracy of 0.9572 on the training queries.

We evaluate three reranking settings — **no_rerank**, **hard_filter**, and **soft_boost** — on four retrieval pipelines: TF-IDF, BM25+, embeddings, and hybrid RRF. Table 12 reports the results at $K = 100$.

Method	Rerank	Precision@100	Recall@100	MRR@100	Query Cat. Acc.
bm25_plus	hard_filter	0.0202	0.3116	0.2611	0.9572
bm25_plus	no_rerank	0.0202	0.3116	0.2697	0.9572
bm25_plus	soft_boost	0.0202	0.3116	0.2700	0.9572
embeddings	hard_filter	0.0333	0.4854	0.3406	0.9572
embeddings	no_rerank	0.0333	0.4854	0.3504	0.9572
embeddings	soft_boost	0.0333	0.4854	0.3488	0.9572
hybrid_rrf	hard_filter	0.0336	0.4918	0.3643	0.9572
hybrid_rrf	no_rerank	0.0336	0.4918	0.3753	0.9572
hybrid_rrf	soft_boost	0.0336	0.4918	0.3759	0.9572
tfidf	hard_filter	0.0173	0.2625	0.1949	0.9572
tfidf	no_rerank	0.0173	0.2625	0.2020	0.9572
tfidf	soft_boost	0.0173	0.2625	0.2031	0.9572

Table 12: Phase 2 reranking results on the full corpus at $K = 100$.

Three results stand out. First, **hard_filter** consistently reduces MRR for all retrieval methods, showing that a strict categorical prior is too aggressive in a corpus where category boundaries are informative but not absolute. Second, **soft_boost** is more stable: it slightly improves TF-IDF (0.2031 vs. 0.2020), BM25+ (0.2700 vs. 0.2697), and hybrid RRF (0.3759 vs. 0.3753), while slightly degrading embeddings (0.3488 vs. 0.3504). Third, the best default configuration in this controlled evaluation is **hybrid_rrf + soft_boost**, with MRR@100 = 0.3759, only marginally above the pure hybrid baseline.

To further analyze the effect of category probabilities, Notebook 08 varies the `boost_factor` for `soft_boost` on embeddings and hybrid RRF. The results are shown in Table 13.

Method	Boost Factor	Precision@100	Recall@100	MRR@100	Query Cat. Acc.
embeddings	-2.0	0.0333	0.4854	0.0422	0.9572
embeddings	-1.0	0.0333	0.4854	0.1414	0.9572
embeddings	-0.5	0.0333	0.4854	0.1538	0.9572
embeddings	0.5	0.0333	0.4854	0.3497	0.9572
embeddings	1.0	0.0333	0.4854	0.3489	0.9572
embeddings	1.5	0.0333	0.4854	0.3488	0.9572
embeddings	2.0	0.0333	0.4854	0.3487	0.9572
hybrid_rrf	-2.0	0.0336	0.4918	0.0467	0.9572
hybrid_rrf	-1.0	0.0336	0.4918	0.1606	0.9572
hybrid_rrf	-0.5	0.0336	0.4918	0.2996	0.9572
hybrid_rrf	0.5	0.0336	0.4918	0.3773	0.9572
hybrid_rrf	1.0	0.0336	0.4918	0.3767	0.9572
hybrid_rrf	1.5	0.0336	0.4918	0.3759	0.9572
hybrid_rrf	2.0	0.0336	0.4918	0.3754	0.9572

Table 13: Sensitivity analysis of `soft_boost` with respect to the `boost_factor`.

Two conclusions emerge. Negative boost factors collapse performance, confirming that the classifier signal is genuinely informative for retrieval. For positive values, gains quickly saturate: for hybrid RRF, the best value is `boost_factor` = 0.5, with `MRR@100` = 0.3773, while stronger boosts bring little or no additional benefit. For embeddings, even the best positive boost remains slightly below the no-rerank baseline.

These results should be interpreted separately from the final Phase 2 submission described in Section 9. Notebook 08 isolates the effect of classification-aware reranking on training queries and therefore uses Logistic Regression to enable `soft_boost`. By contrast, the final end-to-end submission pipeline follows the stable configuration implemented in `src/`, combining TF-IDF + LinearSVC classification, query expansion, pseudo-relevance feedback, hybrid retrieval, and `hard_filter` reranking.

Overall, classification-aware reranking has a measurable but limited impact. `hard_filter` is too rigid for this corpus, whereas `soft_boost` provides a small and more reliable improvement, especially on top of the hybrid retriever.

9 Kaggle Submission Phase 2

9.1 Pipeline Configuration Phase 2

The Phase 2 submission is generated by Notebook 09 together with the end-to-end function `run_phase2_pipeline(...)` implemented in `src/kaggle/submit_phase2.py`. This pipeline combines classification, query expansion, pseudo-relevance feedback, hybrid retrieval, reranking, and Kaggle export within a single reproducible workflow.

The final configuration used in Notebook 09 is summarized in Table 14.

Parameter	Value	Role
<code>RANDOM_SEED</code>	42	global reproducibility
<code>CLASSIFIER_METHOD</code>	<code>svc</code>	final classifier family
<code>FEATURE_METHOD</code>	<code>tfidf</code>	final classification features
<code>RERANK_STRATEGY</code>	<code>hard_filter</code>	category-aware reranking
<code>RERANK_BOOST_FACTOR</code>	0.5	boost strength if <code>soft_boost</code> is used
<code>TOP_K</code>	100	Kaggle submission cutoff
<code>TEXT_FIELD</code>	<code>content</code>	shared input field
<code>EMBEDDING_MODEL_NAME</code>	<code>all-MiniLM-L12-v2</code>	dense retriever encoder
<code>HYBRID_RRF_K</code>	30	fusion constant for RRF
<code>HYBRID_WEIGHT_EMBEDDINGS</code>	3.5	embedding weight in fusion
<code>HYBRID_WEIGHT_BM25</code>	0.5	BM25 weight in fusion
<code>RETRIEVAL_K_MULTIPLIER</code>	10	candidate pool size multiplier
<code>PRF_TOP_K</code>	3	BM25 documents used for PRF
<code>PRF_MAX_TAGS</code>	10	maximum tags appended to each query

Table 14: Main configuration parameters of the Phase 2 submission pipeline.

In practice, the pipeline executes the following sequence:

1. load raw documents, training queries, and test queries;
2. rebuild the `content` field;
3. train the TF-IDF + LinearSVC classifier on documents and training queries;
4. predict one category for each test query;
5. expand queries with category-specific vocabulary;
6. apply pseudo-relevance feedback based on BM25 tags;
7. retrieve an enlarged candidate pool with hybrid BM25 + embeddings;
8. rerank the candidates with `hard_filter`;
9. export the final CSV submission with predicted categories.

Because `TOP_K=100` and `RETRIEVAL_K_MULTIPLIER=10`, the retriever produces 1000 candidates per query before reranking. This larger pool gives the reranker enough room to promote category-consistent documents while preserving fallback candidates from other categories.

Listing 10: Core configuration and execution of the Phase 2 submission pipeline

```
config = Phase2Config(
    classifier_method=CLASSIFIER_METHOD,
    feature_method=FEATURE_METHOD,
    rerank_strategy=RERANK_STRATEGY,
    rerank_boost_factor=RERANK_BOOST_FACTOR,
    top_k=TOP_K,
    text_field=TEXT_FIELD,
    embedding_model_name=EMBEDDING_MODEL_NAME,
    hybrid_rrf_k=HYBRID_RRF_K,
    hybrid_weight_embeddings=HYBRID_WEIGHT_EMBEDDINGS,
    hybrid_weight_bm25=HYBRID_WEIGHT_BM25,
)

submission_df = run_phase2_pipeline(config)
```

It is important to distinguish this final submission pipeline from the controlled reranking analysis of Section 8.4. Section 8.4 studies the isolated effect of **hard_filter** and **soft_boost** on training queries, whereas the final submission follows the stable end-to-end configuration implemented in `src/`: TF-IDF + LinearSVC classification, query expansion, pseudo-relevance feedback, hybrid retrieval, and **hard_filter** reranking.

9.2 Results and Reproducibility

The Phase 2 submission pipeline produces a final CSV file containing one row per test query, a top-100 ranked list of document IDs, and the predicted category of the query. In project mode, the source-of-truth implementation writes this file as `submission.csv`; in the notebook workflow, an equivalent copy may also appear as `submission_phase2.csv`.

A direct inspection confirms that the generated submission:

- contains 141 rows, matching the number of test queries;
- uses the three expected columns `query_id`, `relevant_doc_ids`, and `category`;
- provides exactly 100 document IDs and one predicted category for each query.

With the current configuration, the pipeline processes:

- 216,041 documents;
- 327 labeled training queries;
- 141 test queries;
- 1000 retrieved candidates per query before final reranking.

Reproducibility is ensured by four mechanisms:

- a global seed `RANDOM_SEED = 42` applied through `set_global_seeds(...)`;
- systematic rebuilding of the `content` field from raw files;
- embedding caching in `data/cache/`;
- explicit validation of the Kaggle cutoff, raising a `ValueError` if `top_k` is not equal to 100.

The pipeline also stores a timestamped JSON run descriptor recording the active configuration and output path. Overall, the Phase 2 pipeline is reproducible, format-safe, and fully compatible with Kaggle submission constraints.

10 Conclusion

This project designed, implemented, and evaluated a complete information retrieval engine across two phases, evolving from a classic retrieval pipeline into a classification-enhanced system. A major strength of the work is its modular and reproducible structure: preprocessing, retrieval, evaluation, classification, reranking, and submission generation are implemented as reusable components, which made systematic experimentation possible throughout the project.

In Phase 1, we established a strong retrieval baseline on the StackExchange corpus. We implemented and compared three complementary retrieval families: TF-IDF, BM25/BM25+, and dense semantic embeddings with SentenceTransformers. The experiments showed a clear hierarchy of behaviors: TF-IDF provides a fast and interpretable lexical baseline, BM25 improves lexical ranking through term-frequency saturation and length normalization, and embeddings offer the strongest standalone semantic ranking quality. The best overall submission strategy for this first phase was a hybrid BM25+ + embeddings pipeline combined through Reciprocal Rank Fusion, which successfully balanced exact lexical matching and semantic similarity.

Phase 2 extended this baseline by introducing supervised classification into the retrieval pipeline. A complete ablation study over three feature families and four classifier families identified TF-IDF + LinearSVC as the strongest classification model, with a validation macro-F1 of 0.9852. The classification component also generalized very well from documents to queries, confirming that training on both long document texts and short query texts was an effective strategy. More broadly, the results show that the five dataset categories are sufficiently distinct for category prediction to become a reliable auxiliary signal.

We then studied how this signal could improve retrieval through category-aware reranking. Two strategies were explored: `hard_filter`, which promotes documents whose category matches the predicted query label, and `soft_boost`, which adjusts ranking scores using category probabilities. The controlled experiments on training queries show that hard filtering is often too rigid, while soft boosting yields small but more consistent improvements, especially on top of the hybrid retriever. At the same time, the final Phase 2 submission pipeline follows the stable end-to-end configuration implemented in `src/`: TF-IDF + LinearSVC for category prediction, query expansion, pseudo-relevance feedback, hybrid BM25 + embeddings retrieval, and `hard_filter` reranking.

Overall, the project demonstrates both the effectiveness of hybrid retrieval methods and the practical value, but also the limits, of integrating supervised classification into ranking. The resulting system is complete, reproducible, and competitive in the Kaggle setting. Future work could investigate stronger second-stage rerankers such as cross-encoders, learned fusion strategies, or domain-specific fine-tuning of the embedding model, in order to better exploit the interaction between semantic retrieval and category-aware signals.

Writing Assistance

An artificial intelligence tool (ChatGPT) was used during the writing of this report. It was used to rephrase certain sentences and to check spelling.

Artificial intelligence was also used to help write some formulas in $\text{\LaTeX}$.

However, all intellectual property rights for this document remain exclusively with its authors.